U-Net, VAE, GAN and Stable Diffusion

Study notes for the interview: how each model works, its losses, how to evaluate it, what goes wrong and how to fix it. Concepts and hand calculations, no code.

How to read these notes

- Every model follows the same order: **the problem it solves → intuition → architecture → loss → evaluation → failure modes and fixes → worked numbers**.
- Chapter 1 U-Net and all its variants. Chapter 2 VAE. Chapter 3 GAN (loss types, evaluation, mode collapse). Chapter 4 Stable Diffusion. Chapter 5 comparison and question bank. Chapter 6 sources.
- Numbers marked *(from papers / docs)* are published values. Verify them if you quote them exactly.
- **About images:** my work environment cannot download pictures from the internet, so the figures here are drawn by me. Chapter 6 lists direct links to the original paper figures so you can open them next to these notes.

The big map

Model	Task	Core idea	Loss	How to evaluate
U-Net	Segmentation (also the denoiser inside diffusion)	Encoder-decoder with skip connections: context from the encoder, precise location from the skips	Cross-entropy, Dice, IoU, focal, Tversky, combinations	Dice, IoU, sensitivity / specificity, Hausdorff distance
VAE	Generation + learned latent space	Encode to a distribution, sample, decode; keep the latent space smooth with a KL term	Reconstruction + KL (the negative ELBO)	ELBO / bits per dim, reconstruction PSNR / SSIM / LPIPS, FID of samples
GAN	Sharp image generation	Two networks compete: generator forges, discriminator detects	Minimax, non-saturating, LSGAN, hinge, Wasserstein (+GP), R1	FID, IS, KID, precision / recall, density / coverage
Stable Diffusion	Text-to-image	Learn to remove noise in a compressed latent space, guided by text	MSE on predicted noise (epsilon), in latent space	FID, CLIP score, human preference, prompt-adherence benchmarks

How they connect

- **U-Net inside Stable Diffusion:** the denoiser of Stable Diffusion is a U-Net with time embedding and cross-attention.
- **U-Net inside Pix2Pix (a GAN):** the generator is a U-Net, the discriminator judges 70×70 patches.
- **VAE inside Stable Diffusion:** a KL-regularised autoencoder compresses images 8× in each side so that diffusion runs on small latents.
- **GAN ideas inside the VAE of Stable Diffusion:** the autoencoder is trained with an adversarial patch loss to avoid blur.

1. U-Net and its variants

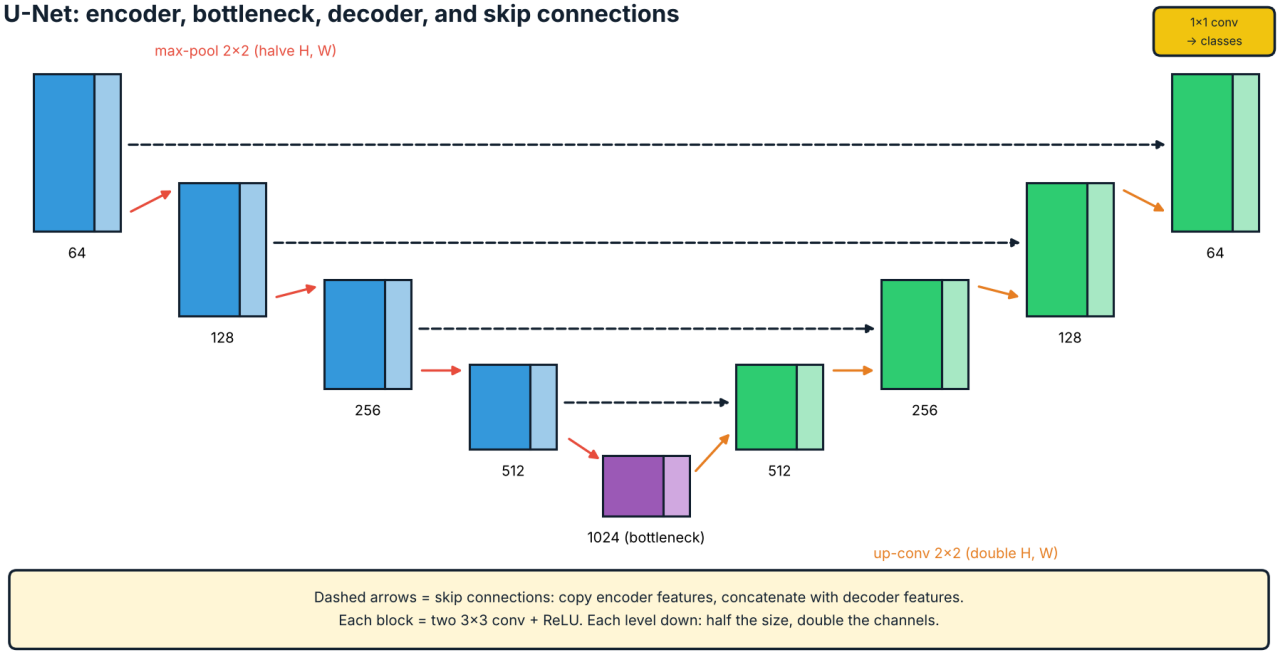
1.1 The problem it solves

- **Classification** gives one label per image. **Segmentation** gives one label per pixel. The output has the same height and width as the input.
- Two needs pull in opposite directions: **context** (what is this object? needs a large field of view, so deep and downsampled layers) and **precise location** (where exactly is the boundary? needs high resolution).
- Earlier approach: run a classifier on patches around every pixel. Slow and wasteful because neighbouring patches overlap.
- FCN (2014) made the network fully convolutional but its output was coarse. **U-Net (2015, Ronneberger, Fischer, Brox)** fixed this with a symmetric decoder and skip connections.
- Original goal: biomedical images, where labelled data is scarce (the paper used only 30 training images for an EM segmentation challenge) and boundaries between touching cells matter.

1.2 Intuition

Think of reading a map. The encoder zooms **out** step by step: you lose detail but you see the whole city and understand what each area is. The decoder zooms **back in**, but it would have forgotten the street details. So at every zoom level it is handed the **original sharp tile** from the way down (the skip connection). The decoder combines 'what it is' (from below) with 'exactly where' (from the skip).

1.3 Architecture of the original U-Net



- **Contracting path (encoder):** repeat 4 times: two 3×3 convolutions + ReLU, then 2×2 max-pooling (stride 2). Channels double each time: 64, 128, 256, 512.
- **Bottleneck:** two 3×3 convolutions with 1024 channels. The receptive field is largest here: each pixel 'sees' a large part of the image.
- **Expanding path (decoder):** repeat 4 times: 2×2 up-convolution (transposed convolution) that doubles height and width and halves channels, **concatenate** with the matching encoder feature map, then two 3×3 convolutions + ReLU.
- **Output:** a 1×1 convolution maps the 64 channels to one score per class per pixel (softmax or sigmoid).
- **Original details *(from the paper)*:** 23 convolutional layers; 3×3 convolutions were **unpadded**, so a 572×572 input gave a 388×388 output (borders are lost); large images are segmented with an **overlap-tile** strategy with mirrored borders. Modern implementations use 'same' padding so input and output sizes match. About 31 million parameters for the standard 64-base version (commonly cited).

- **Why concatenate and not add?** Concatenation keeps both sources separate and lets the next convolution learn how to mix them. Residual-style networks add instead (ResNet-like skips). They are different ideas: U-Net skips cross between encoder and decoder, ResNet skips jump inside a block.

1.4 What makes U-Net train well on small data

- **Heavy augmentation:** random elastic deformations (the paper's key trick), rotations, shifts, intensity changes. Teaches invariance with few images.
- **Skip connections** give a short gradient path to early layers and give the decoder fine detail, so less must be learned from scratch.
- **Weighted loss** forces the network to separate touching cells (explained next).
- **Weight initialisation** with Gaussian of std sqrt(2/N) (He-style) keeps activations balanced, which matters with no batch normalisation in the original (modern versions add batch or instance norm).

1.5 Loss functions for segmentation

Notation: p = predicted probability of foreground for a pixel, g = ground truth (0 or 1), summed over all pixels. TP, FP, FN are true positives, false positives and false negatives counted on pixels.

Cross-entropy and the original weighted version

```
BCE = - (1/N) Σ [ g·log p + (1 - g)·log(1 - p) ]
Original U-Net: E = Σ_x w(x) · log p_l(x)(x) (softmax cross-entropy weighted per pixel)
w(x) = w_c(x) + w0 · exp( - (d1(x) + d2(x))² / (2σ²) ) w0 = 10, σ ≈ 5 pixels
```

- w_c balances class frequencies. d1 and d2 are the distances to the nearest and second-nearest cell border. The exponential term is large **in the thin gap between two touching cells**, so errors there are punished heavily.
- Plain cross-entropy treats every pixel equally. With 2% foreground a network that predicts 'all background' gets 98% accuracy and a low loss: a **class imbalance** problem.

Worked example: border weight

A pixel lies 3 px from one cell and 4 px from another. d1 + d2 = 7, so exponent = -49/(2×25) = -0.98, exp = 0.375. With w_c = 1: **w = 1 + 10×0.375 = 4.75**. A pixel far from borders gets w ≈ 1. The gap pixel counts almost 5× more.

Dice loss (overlap-based, handles imbalance)

```
Dice = 2 Σ p·g / ( Σ p + Σ g ) Dice loss = 1 - Dice
Soft Dice uses the probabilities p directly, so it is differentiable. IoU (Jaccard) loss = 1 -
Σ p·g / ( Σ p + Σ g - Σ p·g )
```

- It measures **overlap relative to object size**, so a small object matters as much as a big one. No class-weight tuning.
- Weakness: gradients can be unstable when the target is tiny or empty (denominator near 0). It is usually combined with cross-entropy: **BCE + Dice** gives stable gradients from BCE and overlap focus from Dice.
- Introduced for segmentation by V-Net (2016); nnU-Net uses Dice + cross-entropy by default.

Worked example: BCE + Dice on 4 pixels

Predictions p = [0.9, 0.8, 0.2, 0.1], truth g = [1, 1, 0, 0].

BCE = -(ln 0.9 + ln 0.8 + ln 0.8 + ln 0.9)/4 = (0.105 + 0.223 + 0.223 + 0.105)/4 = **0.164**.

Dice = 2×(0.9 + 0.8)/(2.0 + 2) = 3.4/4 = 0.85, so Dice loss = **0.15**. Combined = 0.314.

Focal loss (focus on hard pixels)

```
FL = - α_t · (1 - p_t)^γ · log p_t p_t = p if g = 1 else 1 - p γ = 2 is typical
```

Worked example: focal factor

Easy pixel $p_t = 0.9$: CE = 0.105, factor $(0.1)^2 = 0.01$, **FL = 0.001**. Hard pixel $p_t = 0.1$: CE = 2.303, factor $(0.9)^2 = 0.81$, **FL = 1.865**.

Easy pixels are almost ignored (100× smaller) while hard pixels keep most of their loss. Good when most pixels are easy background.

Tversky loss (control false positives vs false negatives)

```
Tversky = TP / ( TP + α·FN + β·FP ) α = β = 0.5 gives Dice; α > β punishes missed pixels more
```

In medical work missing a tumour (FN) is often worse than a false alarm, so $\alpha = 0.7$, $\beta = 0.3$ is common. Other losses worth naming: **Lovász-softmax** (directly optimises IoU), **boundary loss** (distance-based, helps thin structures), **combo loss** (weighted BCE + Dice), deep supervision (loss at several decoder levels).

Loss	Best when	Watch out for
Cross-entropy	Balanced classes; stable baseline	Imbalance: predicts background
Weighted CE	Touching objects; known class frequencies	Needs weight maps / tuning
Dice / IoU	Small foreground, imbalance	Noisy gradients; use with CE
Focal	Many easy pixels, few hard	Extra hyper-parameters γ, α
Tversky	FN and FP have different cost	Tune α, β
Boundary / Lovász	Thin structures, direct metric optimisation	Harder to train from scratch

1.6 Evaluation metrics

```
Dice = 2TP / (2TP + FP + FN) IoU = TP / (TP + FP + FN) Dice = 2·IoU / (1 + IoU)
Precision = TP/(TP+FP) Recall (sensitivity) = TP/(TP+FN) Specificity = TN/(TN+FP) Pixel
accuracy = (TP+TN)/all
```

- **Dice and IoU** measure overlap. Dice ≥ IoU always. Report mean over images and over classes (and say how you average).
- **Pixel accuracy is misleading** with imbalance (see example).
- **Hausdorff distance (HD, HD95):** the largest distance from a point on one boundary to the nearest point on the other. HD95 uses the 95th percentile so a single outlier does not dominate. Measures boundary errors that overlap metrics hide.
- **Average surface distance (ASSD):** mean boundary distance. **Volume difference** matters in clinical use (tumour size).
- **Per-case vs pooled:** compute Dice per image then average. Pooling all pixels hides failures on small cases.
- **Instance-level** metrics (if separate objects matter): panoptic quality, F1 at IoU 0.5 for each object.

Worked example: why accuracy misleads

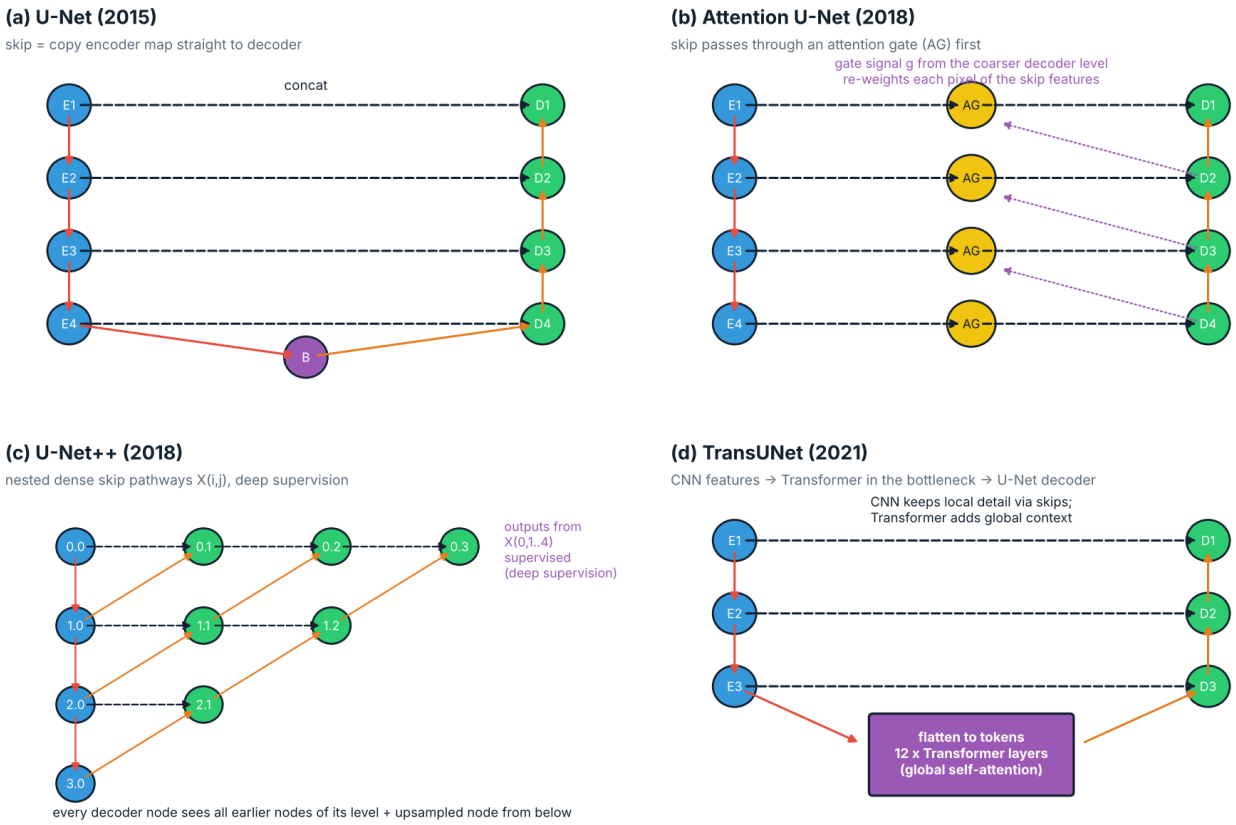
1000 pixels. TP = 80, FP = 20, FN = 10, TN = 890.

Pixel accuracy = (80 + 890)/1000 = **0.97** looks great. Dice = 160/(160 + 20 + 10) = **0.842**. IoU = 80/110 = **0.727**. Precision = 0.80, recall = 0.889, specificity = 0.978.

The foreground is only 9% of the pixels, so accuracy is dominated by easy background.

1.7 The U-Net family: what each variant changes

Variant (year)	What changes	Why	Cost / trade-off
U-Net (2015)	Baseline: conv encoder, conv decoder, concat skips	Context + precise location	Skips pass raw low-level features, including noise
3D U-Net (2016)	All operations in 3D (3D conv, pooling, up-conv)	Volumes: CT, MRI	Much more memory; patches or 2.5D often used
V-Net (2016)	3D, residual blocks, Dice loss	Volumetric segmentation with imbalance	Memory heavy
Residual U-Net (ResUNet)	ResNet-style residual units inside blocks	Deeper nets train more easily	Slightly more compute
R2U-Net (2018)	Recurrent + residual conv units	Better feature accumulation	More compute, modest gains
Dense U-Net	DenseNet blocks (each layer sees all earlier maps)	Feature reuse, fewer parameters per feature	High memory for concatenations
Attention U-Net (2018)	Attention gates on the skip paths	Suppress irrelevant regions, focus on target (pancreas)	Small extra cost; gains largest for small organs
U-Net++ (2018)	Nested, dense skip pathways + deep supervision	Close the semantic gap between encoder and decoder maps; prune at inference	More parameters and training memory
UNet 3+ (2020)	Full-scale skip connections (every decoder level sees all scales)	Use fine and coarse detail together	Fewer parameters than U-Net++, more complex wiring
nnU-Net (2018 / 2021)	Not a new block: a **self-configuring pipeline** (patch size, depth, normalisation, loss, ensembling decided from the dataset)	Most of the gain in practice comes from correct configuration	Needs compute for 2D / 3D / cascade runs; strong default baseline
TransUNet (2021)	CNN features → Transformer encoder in the bottleneck → U-Net decoder	Global context that convolutions lack	Needs pretraining; heavier
Swin-UNet (2021)	Pure Swin Transformer U-shape (patch merging down, patch expanding up)	Long-range modelling with local windows	Data hungry
UNETR (2021)	3D ViT encoder; features from layers 3, 6, 9, 12 feed a CNN decoder via skips	Global context for volumes	Very heavy; data hungry
UNeXt (2022)	Convolutions early, tokenised MLP in the bottleneck	Light and fast	Lower capacity



Attention U-Net in detail

- Problem: the raw skip features carry everything, including irrelevant background. The decoder receives them blindly.
- An **attention gate** takes the skip feature x (fine detail) and a **gating signal g** from the coarser decoder level (it knows what the target roughly is). It produces a map α between 0 and 1 for each pixel and multiplies the skip features by α. Pixels the network considers irrelevant are suppressed before concatenation.
- This is **additive attention** computed with small 1×1 convolutions. It is a local gating, not the global self-attention of Transformers.

```
q = ψᵀ · ReLU( W_x·x + W_g·g + b_g ) + b_ψ α = sigmoid(q) x_hat = α ⊙ x (fed to the decoder)
```

U-Net++ in detail

- Skip connections in U-Net join feature maps that are **semantically far apart**: a very low-level encoder map meets a high-level decoder map. U-Net++ fills the gap with intermediate convolution blocks.
- Node X(i, j): level i, j-th step along the nested path. X(i, 0) is the encoder. X(i, j) = conv(concat of all X(i, k) for k < j , upsampled X(i+1, j-1)). Every node therefore receives **all earlier nodes at its level**.
- **Deep supervision:** a loss is attached to the outputs X(0,1) ... X(0,4). At inference you can **prune**: use only a shallower output for a faster model.
- Reported gains: consistent improvements across several medical datasets *(from the paper)*; the price is more parameters and memory.

nnU-Net: why it matters

- Observation: many 'new architectures' lose against a well-configured plain U-Net. nnU-Net automates the configuration: it looks at the dataset (image sizes, spacing, class frequencies) and sets patch size, batch size, network depth, resampling, normalisation, loss (Dice + cross-entropy), augmentation and post-processing.
- It trains 2D, 3D full-resolution and 3D cascade variants, picks or ensembles the best by cross-validation, and has won many segmentation challenges.
- Interview line: '**before changing the architecture, make sure the training recipe is right**'.

Transformer-based U-Nets

- Convolutions see only a local neighbourhood. Stacked layers grow the field of view slowly, but a Transformer relates **every position to every other** in one layer (global context).
- **TransUNet:** ResNet features are flattened into tokens, a ViT processes them, then a U-Net decoder upsamples using CNN skips. Keeps local precision from the CNN skips.
- **Swin-UNet:** shifted-window attention gives linear cost in image size. Downsampling = patch merging, upsampling = patch expanding.
- **UNETR:** a plain ViT as encoder on 3D patches; its intermediate layers (3, 6, 9, 12) are reshaped into feature maps and passed to a CNN decoder.
- Trade-off: Transformers need more data or pretraining. On small medical datasets a tuned U-Net / nnU-Net is often still the best choice.

1.8 Practical issues and fixes

Problem	Why it happens	Fix
Predicts only background	Class imbalance	Dice / focal / Tversky, class weights, oversample foreground patches
Merged touching objects	No boundary awareness	Weighted loss (original paper), boundary loss, instance post-processing
Overfits with few images	Too many parameters, tiny dataset	Strong augmentation (elastic), pretrained encoder, dropout, smaller U-Net, nnU-Net recipe
Border artefacts	Unpadded conv or tiling	Same padding, overlapping tiles, blending
Out of memory (3D)	Large volumes	Patch-based training, mixed precision, 2.5D, gradient checkpointing
Blurry or noisy edges	Low-resolution output, strong smoothing	Deeper skips, deep supervision, test-time augmentation, CRF / refinement
Good Dice but bad boundaries	Overlap metric hides edge error	Also report HD95, use boundary loss
Poor on new scanner	Domain shift	Intensity augmentation, normalisation per image, fine-tune on target domain

1.9 Where else U-Net appears

- **Diffusion models:** the denoiser takes a noisy image and a timestep and outputs the noise. Same shape in and out, so a U-Net is natural. Adds timestep embedding and attention (Chapter 4).
- **Pix2Pix:** U-Net generator with a PatchGAN discriminator; skip connections let low-level structure (edges) pass straight from input to output.
- **Denoising, super-resolution, inpainting, optical flow:** any task where output is an image aligned with the input.

2. Variational Autoencoder (VAE)

2.1 The problem it solves

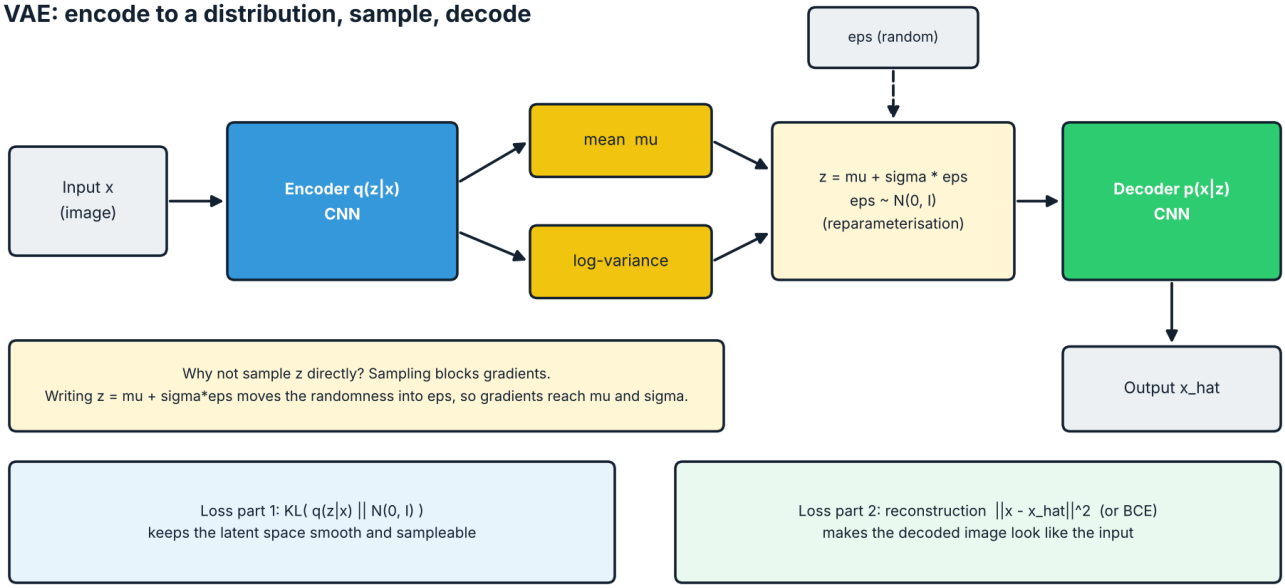
- We want to **generate new data** (images, molecules, sounds) that look like the training data, and also to have a compact **latent code** that captures what matters.
- A plain **autoencoder** (encoder → code → decoder, trained to reconstruct) compresses well but cannot generate: its latent space has holes. Pick a random code and the decoder outputs garbage, because it only learned to decode the specific codes that training images produced.
- A VAE reshapes the latent space so that **nearby codes decode to similar images and random codes from a simple distribution decode to realistic images**.

2.2 Intuition

Instead of mapping an image to one point, the encoder maps it to a **small cloud** (a Gaussian: centre μ and spread σ). During training we sample a point from the cloud and decode it. Because clouds from similar images overlap, the decoder learns that nearby points mean similar things, which fills the holes. A second force (the KL term) pulls all clouds toward one standard bell-shaped region, so at generation time we can sample from that region and decode.

Analogy and its limit: it is like describing a face by sliders. The cloud says 'about this setting, give or take'. The analogy stops being exact because the sliders are learned, not named, and several sliders may mix several properties.

2.3 Architecture



- **Encoder q(z|x):** a CNN (or MLP) that outputs two vectors: the mean μ and the log-variance $\log \sigma^2$. (It predicts the log of the variance so the value can be any real number and σ^2 stays positive.)
- **Sampling with the reparameterisation trick:** $z = \mu + \sigma \odot \varepsilon$ with $\varepsilon \sim N(0, I)$. The randomness is moved to ε, which does not depend on the network weights, so backpropagation flows through μ and σ.
- **Decoder p(x|z):** a CNN that maps z back to image space (transposed convolutions or upsampling). Its output is the mean of a distribution over pixels (Gaussian, or Bernoulli for binary images).
- **Generation:** throw away the encoder. Sample $z \sim N(0, I)$, decode.
- **Interpolation:** encode two images, walk linearly between their μ, decode each point: a smooth morph. This works because the latent space has no holes.

2.4 The loss: the negative ELBO

We want to maximise the likelihood p(x) of the training data but it needs an integral over all z that is intractable. The VAE maximises a **lower bound** on log p(x) called the ELBO (evidence lower bound).

```
log p(x) ≥ E_q(z|x)[ log p(x|z) ] - KL( q(z|x) || p(z) ) (ELBO)
Loss = - ELBO = Reconstruction loss + KL term p(z) = N(0, I)
```

Where the ELBO comes from (short derivation)

- Start from log p(x) = log ∫ p(x|z) p(z) dz. Multiply and divide by q(z|x) inside the integral.
- By Jensen's inequality (log of an average ≥ average of the log) we move the log inside: log p(x) ≥ E_q[log p(x|z) + log p(z) - log q(z|x)].
- Rearranging gives reconstruction term minus KL(q(z|x) || p(z)).
- The gap between log p(x) and the ELBO is exactly KL(q(z|x) || p(z|x)): how far the encoder's guess is from the true posterior. Better encoder → tighter bound.

The two terms

- **Reconstruction:** with a Gaussian decoder it is ||x - x_hat||² (mean squared error up to a constant). With Bernoulli pixels it is binary cross-entropy. 'Make the output look like the input.'
- **KL:** penalises encoder clouds that differ from N(0, I). 'Keep the latent space organised and sampleable.'
- They **fight**: reconstruction wants distinct, narrow, informative clouds (a lot of information in z); KL wants everything to look the same (no information). The balance decides the behaviour.

```
KL( N(μ, σ²) || N(0, 1) ) per dimension = - 1/2 · ( 1 + log σ² - μ² - σ² ) sum over latent dimensions
```

Worked example: closed-form KL for one latent dimension

μ = 0, σ² = 1 (already the prior): KL = -½(1 + 0 - 0 - 1) = **0**.

μ = 1, σ² = 1 (shifted): KL = -½(1 + 0 - 1 - 1) = **0.5**. Shifting the centre costs.

μ = 0, σ² = 4 (too wide): KL = -½(1 + ln 4 - 0 - 4) = -½(1 + 1.386 - 4) = **0.807**.

μ = 2, σ² = 0.25 (informative, narrow): KL = -½(1 + ln 0.25 - 4 - 0.25) = **2.32**. Information in z costs KL: this is the price of encoding 'a lot' in a dimension.

Why the KL term has a real meaning

- Average KL over the dataset is an upper bound of the **bits of information** that z carries about x. KL close to 0 means z carries almost nothing.
- Per-dimension KL tells you which latent dimensions are **active** (KL clearly above 0) and which are unused.

2.5 Training in practice

- Loss per image = reconstruction (sum over pixels) + β · KL (sum over latent dimensions). Standard VAE has β = 1.
- Adam optimiser, learning rate around 1e-3 for small models. Normalise pixels to [0, 1] and use BCE, or to [-1, 1] and use MSE.
- **Scale matters:** if reconstruction is averaged over pixels but KL summed over dimensions, the balance is wrong. Keep the same convention (sum or mean) for both, or tune β.
- Monitor **reconstruction and KL separately**, not only the total.

2.6 Known problems, their causes and fixes

Blurry samples

- Cause: the Gaussian decoder with MSE predicts the **average** of all images compatible with a code, and an average of sharp images is blurry. Also KL pushes the clouds to overlap, mixing different images.
- Fixes: perceptual (LPIPS) loss instead of pixel loss; add an **adversarial loss** (VAE-GAN, VQGAN, Stable Diffusion's autoencoder); more expressive decoder; hierarchical VAEs (NVAE); or use the VAE only as a compressor and generate with a diffusion model in its latent space (Stable Diffusion).

Posterior collapse

- **What:** the encoder ignores the input: q(z|x) ≈ p(z) = N(0, I) for every x. KL ≈ 0. The decoder generates the 'average' output regardless of z. Samples all look alike or generic.

- **Why:** if the decoder is powerful enough to model the data on its own (autoregressive decoders such as LSTMs or PixelCNN, very strong decoders), using z is unnecessary and costs KL, so the optimiser sets z aside. It is also a local optimum early in training when the encoder is poor.
- **How to detect:** total KL near 0; per-dimension KL all near 0 (no 'active units'); decoding different z gives the same output; mutual information between x and z near 0.
- **Fixes:** (1) **KL annealing / warm-up**: start β at 0 and increase to 1 over the first epochs, so the model first learns to use z. (2) **Free bits**: do not penalise KL below a small threshold λ per dimension. (3) **Weaken the decoder** (smaller, word dropout in text, limited context). (4) **Lower β** (below 1) trading sampling quality for information. (5) Train the encoder more aggressively than the decoder at the start. (6) Use a **discrete latent** (VQ-VAE), which has no KL term to collapse. (7) Add skip paths from z to every decoder layer.

Holes and mismatch between aggregate posterior and prior

- Even with a good KL, the average of all encoder clouds may not match N(0, I) exactly, so some prior samples land in regions the decoder has never seen. Fixes: more expressive priors (mixtures, flows, learned prior), hierarchical VAEs, or a second generative model on the latents (as in Stable Diffusion and VQ-VAE + Transformer).

2.7 Important variants

Variant	Idea	Loss change	Why use it
β-VAE (2017)	Multiply the KL by β > 1	Recon + β·KL	Pushes toward **disentangled** factors (one latent per factor of variation) at the cost of blurrier output
Conditional VAE	Feed a label or condition to encoder and decoder	Same ELBO, conditioned	Controlled generation (digit class, text)
VQ-VAE (2017)	Latent is **discrete**: pick the nearest vector from a learned codebook	Recon + codebook loss + commitment loss	No posterior collapse; sharper; works with autoregressive priors (images, audio)
VQ-VAE-2	Hierarchy of discrete latents	Same	High-resolution images
VQGAN (2021)	VQ-VAE + perceptual + adversarial loss; Transformer prior	Recon (perceptual) + GAN + codebook	Sharp reconstructions; basis for latent generative models
Hierarchical VAE (NVAE, VDVAE)	Many layers of latents, coarse to fine	ELBO with a KL per layer	Competitive likelihoods and sharper samples
KL-regularised AE (Stable Diffusion)	Tiny KL weight (about 1e-6) + LPIPS + patch adversarial loss	Recon + LPIPS + GAN + very small KL	A **compressor**, not a generator: 8× smaller in each side, little blur

VQ-VAE in a bit more detail

- Encoder output z_e is replaced by the nearest of K codebook vectors e_k. The decoder receives the code vector.
- The nearest-neighbour step is not differentiable, so the gradient is copied from the decoder input straight to the encoder output (**straight-through estimator**).
- Loss = $||x - \hat{x}||^2 + ||sg[z_e] - e||^2 + \beta \cdot ||z_e - sg[e]||^2$ where sg is stop-gradient. The second term moves codebook vectors toward encoder outputs, the third (commitment, $\beta \approx 0.25$) keeps the encoder from drifting away from the codes.
- **Codebook collapse** (only a few codes used): fix with EMA codebook updates, k-means initialisation, restarting dead codes, smaller codebook dimension, or better normalisation.

2.8 How to evaluate a VAE

- **ELBO / negative log-likelihood** on held-out data, often given in **bits per dimension** (nats divided by (number of dimensions × ln 2)). Lower is better. Report the reconstruction and KL parts separately.
- **Reconstruction quality:** MSE, PSNR, SSIM, LPIPS (perceptual) between input and reconstruction. For a compressor (Stable Diffusion's VAE) also **rFID** (FID of reconstructions).
- **Sample quality and diversity:** FID of samples drawn from the prior against real data (see Chapter 3).

- **Latent space health:** number of active units (dimensions with variance of μ across data above a threshold), KL per dimension, interpolation smoothness, latent traversals by eye.
- **Disentanglement** (β-VAE style work): MIG, DCI, FactorVAE score, which need datasets with known factors.
- **Downstream:** accuracy of a classifier trained on the latent codes.

Worked example: bits per dimension

A binarised-MNIST VAE reports a negative ELBO of 90 nats per image. Dimensions = 28×28 = 784. Bits per dimension = 90/(784 × ln 2) = 90/543.4 = **0.166**.

Comparing models on different data needs this normalisation. Remember that ELBO is a bound: the true likelihood is at least as good.

2.9 VAE versus plain autoencoder in one table

	Autoencoder	VAE
Encoder output	One point	Mean and variance (a cloud)
Latent space	May have holes	Smooth and organised by the KL term
Generation	Not reliable	Sample from N(0, I) and decode
Loss	Reconstruction	Reconstruction + KL (negative ELBO)
Typical issue	Overfits to identity mapping	Blur, posterior collapse

3. Generative Adversarial Networks (GANs)

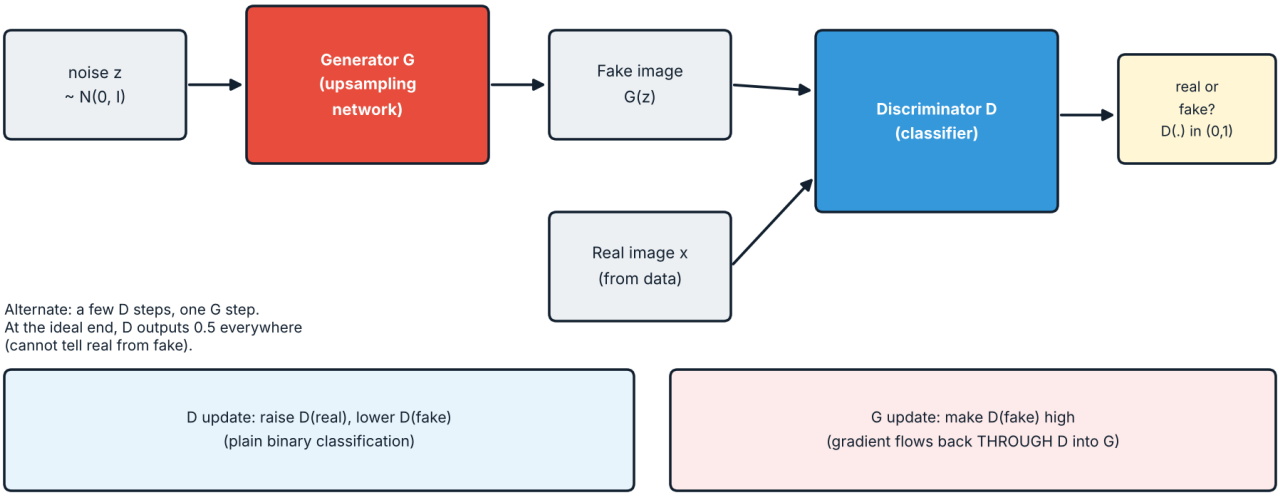
3.1 The problem it solves

- Generate realistic samples (images, video, audio) from a simple random vector, **without writing down a likelihood**.
- VAEs optimise pixel-wise reconstruction, which averages and blurs. A GAN replaces 'is the pixel close?' with a **learned judge** that asks 'does this look real?', which rewards sharp, realistic detail.
- Goodfellow et al. 2014. Dominated image generation until diffusion models (2020 to 2022).

3.2 Intuition

A counterfeiter (**generator G**) makes fake banknotes. A detective (**discriminator D**) tries to tell fake from real. Each improves because of the other. When training works, the fakes become so good that the detective guesses 50/50. Where the analogy breaks: the generator never sees the real notes. It only learns from the detective's feedback (the gradient through D).

GAN: a forger (G) and a detective (D) trained against each other



3.3 Architecture

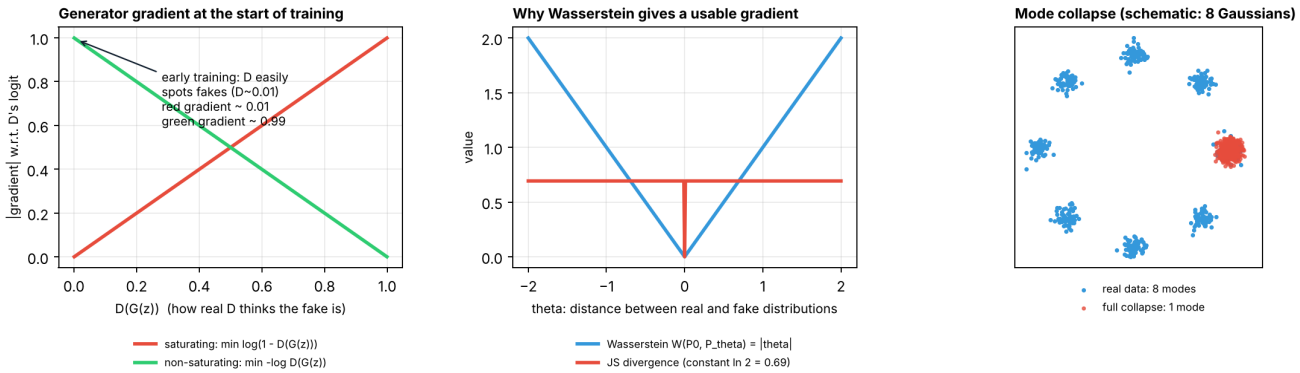
- **Generator:** noise vector z (often 100 to 512 numbers from N(0, I)) → fully connected → reshape → a stack of upsampling blocks (transposed convolution or upsample + convolution) → image. Last activation tanh (images in [-1, 1]).
- **Discriminator:** an ordinary image classifier: strided convolutions that downsample, ending in one score (probability of real, or an unbounded score in WGAN).
- **DCGAN guidelines (2015)** that made GANs train: strided convolutions instead of pooling, batch normalisation, no fully connected layers in the body, ReLU in G and LeakyReLU in D.
- **Training loop:** (1) train D on a batch of real and a batch of fake (G frozen). (2) train G so that D calls its fakes real (D frozen, but gradients flow **through** D to G). Repeat. WGAN-GP trains D 5 times per G step.

3.4 The objective and what it really optimises

```
min_G max_D V(D, G) = E_x~p_data [ log D(x) ] + E_z~p_z [ log( 1 - D(G(z)) ) ]
For a fixed G the best discriminator is D*(x) = p_data(x) / ( p_data(x) + p_g(x) )
Plugging D* in: V(D*, G) = - log 4 + 2 · JSD( p_data || p_g ) minimum -log 4 = -1.386 when p_g
= p_data (then D* = 0.5 everywhere)
```

- So an ideal GAN minimises the **Jensen-Shannon divergence** between real and generated distributions.
- **Problem:** real images and generated images live on thin low-dimensional surfaces inside pixel space and usually **do not overlap**. When two distributions do not overlap, JSD is the constant ln 2 no matter how far apart they are, so there is **no useful gradient** (see the plot below). This is the theoretical root

of unstable GAN training.



Left: why the generator uses a different loss than the textbook one. Middle: why Wasserstein distance helps. Right: schematic of mode collapse (a made-up 2-D example with 8 Gaussians). Next sections explain each.

3.5 Loss functions, one by one

1. Original (saturating) GAN loss

```
D loss: - [ log D(x) + log( 1 - D(G(z)) ) ] G loss: + log( 1 - D(G(z)) ) (minimise)
```

2. Non-saturating GAN loss (what people actually use)

```
G loss: - log D(G(z))
```

- Early in training D rejects fakes confidently: D(G(z)) ≈ 0. For the saturating loss the gradient with respect to D's logit is proportional to D(G(z)) ≈ 0, so **G learns almost nothing exactly when it needs to learn most** (vanishing gradient). The non-saturating loss has gradient proportional to 1 - D(G(z)) ≈ 1.

Worked example: saturation with numbers

D(x) = 0.9 on a real image, D(G(z)) = 0.2 on a fake. D loss = -(ln 0.9 + ln 0.8) = 0.105 + 0.223 = **0.329**. Non-saturating G loss = -ln 0.2 = **1.609**. Saturating G loss = ln 0.8 = -0.223.

Early training: D(G(z)) = 0.01. Gradient w.r.t. the logit a (D = sigmoid(a)): saturating = D = **0.01**, non-saturating = 1 - D = **0.99**. About a 100× difference.

3. Least-squares GAN (LSGAN)

```
D loss: ½ E[ (D(x) - 1)² ] + ½ E[ D(G(z))² ] G loss: ½ E[ (D(G(z)) - 1)² ]
```

Replaces the log loss by squared error. Fakes that are on the correct side of the boundary but far from the real data still receive gradient (no saturation), which gives more stable training and slightly sharper images.

Worked example: LSGAN

D(x) = 0.9, D(G(z)) = 0.2: D loss = ½(0.9 - 1)² + ½(0.2)² = 0.005 + 0.02 = **0.025**. G loss = ½(0.2 - 1)² = **0.32**.

4. Hinge loss (SAGAN, BigGAN)

```
D loss: E[ max(0, 1 - D(x)) ] + E[ max(0, 1 + D(G(z))) ] G loss: - E[ D(G(z)) ] (D outputs raw scores, not probabilities)
```

D is only pushed until real scores exceed +1 and fake scores fall below -1; beyond that margin there is no more gradient, so D does not become overconfident.

Worked example: hinge

Real score 0.7, fake score -0.3: real term max(0, 1 - 0.7) = 0.3; fake term max(0, 1 + (-0.3)) = 0.7. D loss = **1.0**. If the real score were 1.5 and the fake -1.5, D loss = **0**.

5. Wasserstein GAN (WGAN) and WGAN-GP

```
W(P_r, P_g) = sup over 1-Lipschitz f of E_x~P_r[ f(x) ] - E_x~P_g[ f(x) ]
(Kantorovich-Rubinstein duality)
Critic loss: E[ f(G(z)) ] - E[ f(x) ] Generator loss: - E[ f(G(z)) ]
WGAN-GP adds: λ · E_x_hat[ ( || ∇ f(x_hat) ||₂ - 1 )² ] x_hat = random point on the line
between a real and a fake sample; λ = 10
```

- The 'discriminator' becomes a **critic** that outputs an unbounded score (no sigmoid). The loss approximates the **Earth Mover (Wasserstein-1) distance**: the minimum 'work' to move the generated distribution onto the real one.
- **Why it is better:** Wasserstein distance decreases smoothly as distributions approach, even when they do not overlap (middle plot). Gradients stay informative, and the critic loss **correlates with sample quality**, so you can watch it as a progress signal (a rare property for GANs).
- The critic must be **1-Lipschitz**. WGAN enforced it by clipping weights to [-0.01, 0.01] (crude, causes poor capacity use). **WGAN-GP** instead penalises the gradient norm at points between real and fake samples to be 1. Settings from the paper: $\lambda = 10$, 5 critic steps per generator step, Adam with lr 1e-4, $\beta1 = 0$, $\beta2 = 0.9$. Do not use batch norm in the critic (it couples samples in a batch); use layer norm.
- **Spectral normalisation (SN-GAN)** is a cheaper way to enforce Lipschitz continuity: divide each weight matrix by its largest singular value (estimated by power iteration).

Worked example: WGAN numbers

Critic scores: real 2.1, fake -0.4. Critic loss = -(2.1 - (-0.4)) = **-2.5**, so the estimated Wasserstein distance ≈ 2.5.

Gradient penalty: gradient norm at an interpolated point = 1.5. Penalty = 10 × $(1.5 - 1)^2$ = **2.5**. A norm of exactly 1 gives 0.

Worked example: Wasserstein vs JSD

Real distribution: a point mass at 0. Fake distribution: a point mass at θ. For any $\theta \neq 0$ the JSD is the constant ln 2 = 0.693 (no signal about direction). The Wasserstein distance is $|\theta|$: shrinking θ always reduces it, so the generator has a gradient everywhere.

6. Relativistic GAN

Asks 'is this real sample **more realistic than** the average fake?' instead of 'is it real?'. Often more stable and gives better FID in practice (used in several modern GANs).

7. Regularisers and auxiliary losses

```
R1 penalty: γ/2 · E_x~p_data [ || ∇ D(x) ||² ] (StyleGAN2: lazy regularisation, γ depends on
resolution)
Pix2Pix: G* = argmin_G max_D L_cGAN(G, D) + λ · E || y - G(x) ||₁ λ = 100
CycleGAN cycle consistency: E || F(G(x)) - x ||₁ + E || G(F(y)) - y ||₁ λ = 10
Feature matching: match the mean of an intermediate D feature of fakes to that of reals.
Perceptual loss: distance in VGG / LPIPS feature space.
```

- **R1** penalises D's gradient on real data, stopping D from becoming too sharp around real samples; it makes training locally convergent.
- **L1 reconstruction** (Pix2Pix) anchors the output to the paired target so the GAN term only has to add sharpness.
- **Cycle consistency** makes unpaired translation (horse ↔ zebra) possible: translating there and back must give the original.

Loss	D output	Strength	Weakness
Minimax / non-saturating	Probability	Simple, works with care	Vanishing gradients; unstable
LSGAN	Score	No saturation; stable	Can over-penalise confident correct samples
Hinge	Score	Margin; popular in big models	Needs normalisation tricks
WGAN	Score (critic)	Meaningful loss curve; gradients everywhere	Weight clipping hurts

Loss	D output	Strength	Weakness
WGAN-GP	Score (critic)	Stable; no clipping	Slower (gradient of gradient); no batch norm
Relativistic	Score difference	Stable, good FID	Slightly more complex
R1 / R2 penalty	Added to any loss	Local convergence	Weight must be tuned

3.6 Major GAN families (what each adds)

Model	Key idea	Why it matters
DCGAN (2015)	Convolutional architecture rules	First stable image GAN recipe
Conditional GAN (2014)	Feed a label to G and D	Controlled generation
InfoGAN	Maximise mutual information between a code and the output	Unsupervised disentangled controls
Pix2Pix (2016)	U-Net generator, PatchGAN discriminator, cGAN + L1	Paired image translation
CycleGAN (2017)	Two generators + cycle consistency	Unpaired translation
SRGAN / ESRGAN	Perceptual + adversarial loss	Realistic super-resolution
ProGAN (2017)	Grow resolution progressively; minibatch standard deviation; pixel norm	First 1024×1024 faces; the std layer fights mode collapse
SAGAN (2018)	Self-attention + spectral norm + hinge	Long-range structure
BigGAN (2018)	Large batches, class conditioning, truncation trick	High-fidelity ImageNet; shows scale helps
StyleGAN 1 / 2 / 3	Mapping network, style injection, noise inputs; weight demodulation; alias-free	State-of-the-art faces; controllable and disentangled latents

3.7 Why GAN training is hard

- **Two networks, one moving target:** each loss depends on the other network, so the 'landscape' changes every step. It is a game, not a single minimisation; simultaneous gradient descent can **cycle** or oscillate instead of converging.
- **Vanishing gradients:** a D that is too good gives G nothing to learn from.
- **Non-overlapping distributions** (JSD plateau).
- **Sensitivity** to architecture, learning rates, normalisation and initial conditions.
- **No single loss that tells quality:** the loss values of G and D are not a quality measure (except WGAN critic). You must judge with samples and metrics (Section 3.9).

3.8 Mode collapse

What it is

- The generator produces **only a few kinds of output** (low diversity) even though the real data has many. In the 8-Gaussian toy: real data has 8 clusters, the generator covers only 1 or 2.
- **Full collapse:** the same image for any z. **Partial collapse / mode dropping:** some classes or styles are missing (for example only certain digits or faces without glasses). **Intra-mode collapse:** variety is lost inside a mode.
- It is not the same as non-convergence: collapsed models can have a 'good-looking' loss and sharp images that are all alike.

Why it happens (the mechanism)

- G is rewarded for fooling D, not for covering the data. If one output is the most convincing to the current D, G moves all its mass there (the maximum-seeking behaviour of the min-max order).
- D, seeing only individual samples, can punish that point but cannot see that **all** samples are identical. D then pushes G away from the point; G jumps to another point that D currently likes. The cycle repeats: **whack-a-mole** instead of covering all modes.

- Each sample is judged **independently**: nothing in the plain objective measures the diversity of a batch.
- Contributing factors: overpowered or underpowered D, vanishing gradients (JSD plateau), poor learning-rate balance, small or unbalanced datasets, a very low-capacity or badly regularised G, and too aggressive training of G on a stale D.

How to detect it

- **Look at a large grid of samples** from different z. Near-duplicates, or the same pose / colour, are the signature.
- **Interpolate** between z values. If the output barely changes or jumps between a few images, suspect collapse.
- **Recall (improved precision / recall) or coverage:** low recall with high precision = good-looking but not diverse (see 3.9).
- **Mode counting** on synthetic benchmarks: 8 Gaussians (how many are covered), **Stacked MNIST** (1000 modes: how many of the 1000 digit triples appear; plus KL between generated and true mode frequencies).
- **Diversity measures:** mean pairwise LPIPS or SSIM across generated samples (low LPIPS = collapse), number of statistically different bins (NDB), classifier-based class histogram of the generated images.
- **Training signs:** D loss near 0 while G loss rises, G loss oscillating strongly, FID that stops improving or gets worse while images look sharp.

How to fix it (ordered by how I would try them)

Remedy	How it helps	Used in
Switch the objective to WGAN-GP, or LSGAN / hinge	Gradients stay informative when distributions do not overlap; reduces over-confident D	WGAN-GP, SAGAN, BigGAN
Spectral normalisation in D (and G)	Limits how fast D can change, avoids a D that is too sharp	SN-GAN, SAGAN, BigGAN
Minibatch discrimination / minibatch standard deviation layer	D sees **statistics of the whole batch**, so a batch of identical fakes is easy to spot, and G is forced to diversify	Improved-GAN, ProGAN, StyleGAN
Unrolled GAN	G is updated using D **after** several future D steps, so G cannot simply exploit the current D	Metz et al. 2016
Two time-scale update rule (TTUR)	Different learning rates for D and G (for example D 4e-4, G 1e-4) so D tracks G and training converges	SAGAN, BigGAN
R1 / R2 gradient penalty	Stabilises D near the data, local convergence	StyleGAN2
Packing (PacGAN)	D classifies **several samples at once** (packed together), so lack of variety is penalised	PacGAN
Feature matching	G matches batch-level feature statistics instead of only fooling D	Improved-GAN
Mode-seeking or diversity regulariser	Directly penalises two different z mapping to similar images (maximise image distance / latent distance)	MSGAN
Multiple generators or mixtures	Each generator covers a different mode, extra loss to keep them apart	MGAN, MAD-GAN
Add a reconstruction / encoder (VAE-GAN, VEEGAN, BiGAN)	The encoder forces every real sample to have a place in latent space	VEEGAN, BiGAN
Data augmentation for D (DiffAugment, ADA) and more data	Stops D from memorising a small dataset, which otherwise pushes G to a few safe outputs	StyleGAN2-ADA
Label smoothing, instance noise	Softens D's targets so it does not become overconfident	Improved-GAN
Conditioning on labels; truncation trick for sampling	Conditioning guarantees class coverage; truncation trades diversity for quality at sampling time, so it is a trade-off to report	BigGAN
Better balance of D and G capacity and steps	A D that is too strong gives vanishing gradients; too weak gives no signal	General practice

- **Checklist when I see collapse:** (1) confirm with a sample grid and a diversity / recall measure; (2) check data (balanced, enough, preprocessed to [-1, 1]); (3) move to hinge or WGAN-GP with spectral norm; (4) add minibatch-stddev layer; (5) tune learning rates (TTUR) and D / G balance; (6) add augmentation for D; (7) if it still fails, consider a diffusion model, which does not suffer from mode collapse by construction (it is trained with a likelihood-style objective that covers the whole data distribution).
- **Honest note:** no fix is guaranteed. Mode collapse is reduced, not eliminated, and often a combination is needed.

3.9 Evaluating GANs

There is no likelihood, so we compare **sets** of samples. Good metrics should reward both **fidelity** (each sample looks real) and **diversity** (all modes covered).

Metric	What it measures	Formula idea	Weakness
Inception Score (IS)	Quality and class diversity	exp(E_x KL(p(y|x) || p(y))) with an ImageNet classifier	Needs ImageNet-like content; ignores real data; can be gamed
FID	Distance between real and generated feature distributions	Fit Gaussians to Inception features; Fréchet distance	Gaussian assumption; biased with few samples; ImageNet features
KID	Same idea without Gaussian assumption	Unbiased squared MMD with a cubic polynomial kernel	Higher variance; slower
Precision / Recall (improved)	Fidelity and diversity separately	k-NN balls around real / fake features; fraction of fakes inside real manifold, and reals inside fake manifold	Sensitive to outliers and k
Density / Coverage	Robust version of precision / recall	Counts how many real-neighbourhoods contain each fake	Needs feature extractor
LPIPS diversity	Variety of generated outputs	Mean pairwise LPIPS between samples	Does not measure realism
Human study	Perceived realism	Two-alternative forced choice, ELO	Costly, subjective

```
FID = || μ_r - μ_g ||² + Tr( Σ_r + Σ_g - 2 (Σ_r Σ_g)^(1/2) ) (μ, Σ of 2048-d Inception-v3 pool features; lower is better)
IS = exp( E_x [ KL( p(y|x) || p(y) ) ] ) (between 1 and the number of classes; higher is better)
KID = unbiased MMD² with k(a, b) = ( aᵀb / d + 1 )³
```

Worked example: FID in one dimension

Real features: mean 0, std 1. Generated: mean 1, std 2. FID = $(1 - 0)^2 + (1 + 4 - 2 \times 1 \times 2) = 1 + 1 =$ **2**. The first part measures the shift of the mean, the second the mismatch of the spread. A collapsed generator has a too-small spread, which raises the second term.

Worked example: Inception Score, two classes

Every generated image is classified confidently (p(y|x) is one-hot) and the two classes appear equally often: p(y) = [0.5, 0.5]. KL(one-hot || uniform) = ln 2 for every image, so IS = exp(ln 2) = **2** (the maximum for 2 classes).

If the generator collapsed to one class: p(y|x) = p(y) = one-hot, KL = 0, IS = exp(0) = **1** (the minimum).

Worked example: precision and recall

Generate 100 samples: 80 fall inside the real-data manifold, so precision = **0.80**. Take 100 real samples: only 30 fall inside the generated manifold, so recall = **0.30**.

High precision with low recall is exactly the signature of **mode dropping**: the images are good but many kinds of real image are never produced. FID alone would only show one mixed number.

- **Practical rules for FID:** use at least 10,000 samples (often 50,000), the same preprocessing for real and fake, the same Inception network and resize method (use a standard implementation such as

Clean-FID), and report the real-data subset size. FID depends on the Inception network trained on ImageNet, so for medical or very different domains consider domain features, and do not compare numbers across papers with different protocols.

- **Use several metrics together:** FID (overall), precision / recall or density / coverage (separating quality from diversity), and look at samples.
- **For conditional models:** add class-conditional accuracy of a classifier on generated images, and per-class FID.

3.10 GAN versus VAE versus diffusion (preview)

	VAE	GAN	Diffusion
Sample sharpness	Blurry (pixel loss)	Sharp	Sharp
Diversity / mode coverage	Good	Risk of mode collapse	Very good
Training	Stable	Unstable, needs care	Stable
Sampling speed	One pass (fast)	One pass (fast)	Many steps (slow, improving)
Likelihood	Lower bound (ELBO)	None	Lower bound (training objective)
Latent space	Explicit, encoder available	No encoder by default	Noise levels; inversion possible

4. Stable Diffusion

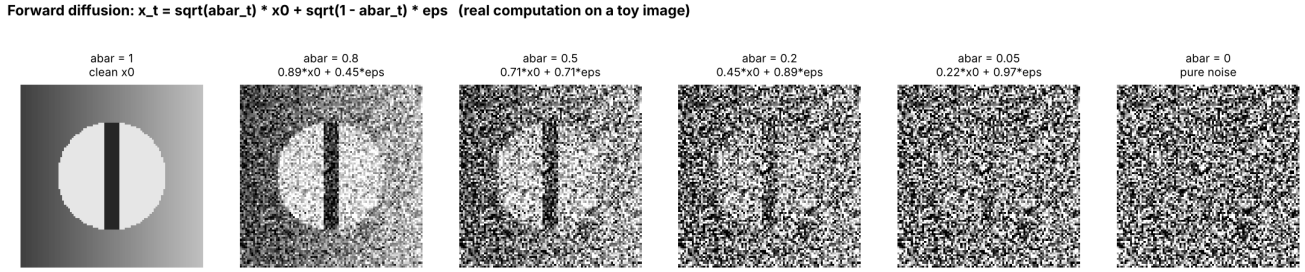
4.1 The problem it solves

- Generate an image that matches a **text prompt**, with high quality and diversity.
- Earlier diffusion models (DDPM, ADM, Imagen) ran in **pixel space**: a U-Net on 512×512×3 images, hundreds of steps. Powerful but extremely expensive.
- **Latent Diffusion (Rombach et al., 2022)** idea: most pixels carry imperceptible detail. First **compress** the image with an autoencoder, then run diffusion only on the small latent. Stable Diffusion is the open model built on this idea.

Compression example: a 512×512×3 image has 786,432 numbers. Its latent is 64×64×4 = 16,384 numbers, **48× fewer**. Self-attention at the 64×64 level compares 4,096 positions: $4{,}096^2 \approx 16.8$ million pairs per head. In pixel space (262,144 positions) it would be about 6.9×10^{10} pairs. This is why latent diffusion made high-resolution text-to-image practical.

4.2 Diffusion from scratch (intuition first)

Take a photograph and add a little random noise, repeatedly, until only static remains. That is easy and needs no learning. Diffusion models learn the **reverse**: given a noisy image, predict the noise that was added, subtract a bit of it, and repeat. Starting from pure static and repeating this removal many times yields a new image. Analogy: a sculptor revealing a statue by removing material step by step; the analogy is limited because here the 'sculptor' is a trained network and the removal is random-guided.



Forward process (fixed, no learning)

```
q(x_t | x_{t-1}) = N( sqrt(1 - β_t) · x_{t-1}, β_t · I ) small noise β_t added at each of T = 1000 steps
α_t = 1 - β_t ᾱ_t = α_1 · α_2 · ... · α_t
x_t = sqrt(ᾱ_t) · x_0 + sqrt(1 - ᾱ_t) · ε ε ~ N(0, I) (jump to any step t in one shot)
```

- ᾱ_t falls from 1 (clean) to near 0 (pure noise). Stable Diffusion uses a **scaled-linear** schedule of β from 0.00085 to 0.012 over 1000 steps *(from the model configuration)*.
- Known subtlety: with this schedule ᾱ_T is small but not exactly 0, so a trace of the original image survives at the last step; this is one reason SD v1 models struggle to produce very dark or very bright images, and 'zero terminal SNR' rescaling is a recommended fix.

Worked example: one noising step

ᾱ_t = 0.64, so sqrt(ᾱ_t) = 0.8 and sqrt(1 - ᾱ_t) = 0.6. One pixel x_0 = 1.0 with noise ε = 0.5: x_t = 0.8×1.0 + 0.6×0.5 = **1.1**.

A perfect noise predictor would give back x_0 = (x_t - 0.6 × ε)/0.8 = (1.1 - 0.3)/0.8 = **1.0**. This identity is how the model's noise prediction is turned into an image estimate.

Training (what the network learns)

- Pick a clean training sample x_0 and a random timestep t. Sample noise ε. Build x_t with the formula above.
- The network ε_θ(x_t, t) predicts the noise. Loss = mean squared error between true and predicted noise. That is all.
- Why predicting the noise works: it is equivalent to learning the **score** (the gradient of the log-density) of the noisy data, which tells in which direction the data becomes more probable. The loss is also a re-weighted form of the ELBO.

```
L_simple = E_{x_0, ε, t} [ || ε - ε_θ( x_t, t ) ||² ] Stable Diffusion: E [ || ε - ε_θ( z_t, t, τ(y) ) ||² ] (z = latent, τ(y) = text embedding)
```

Sampling (generation)

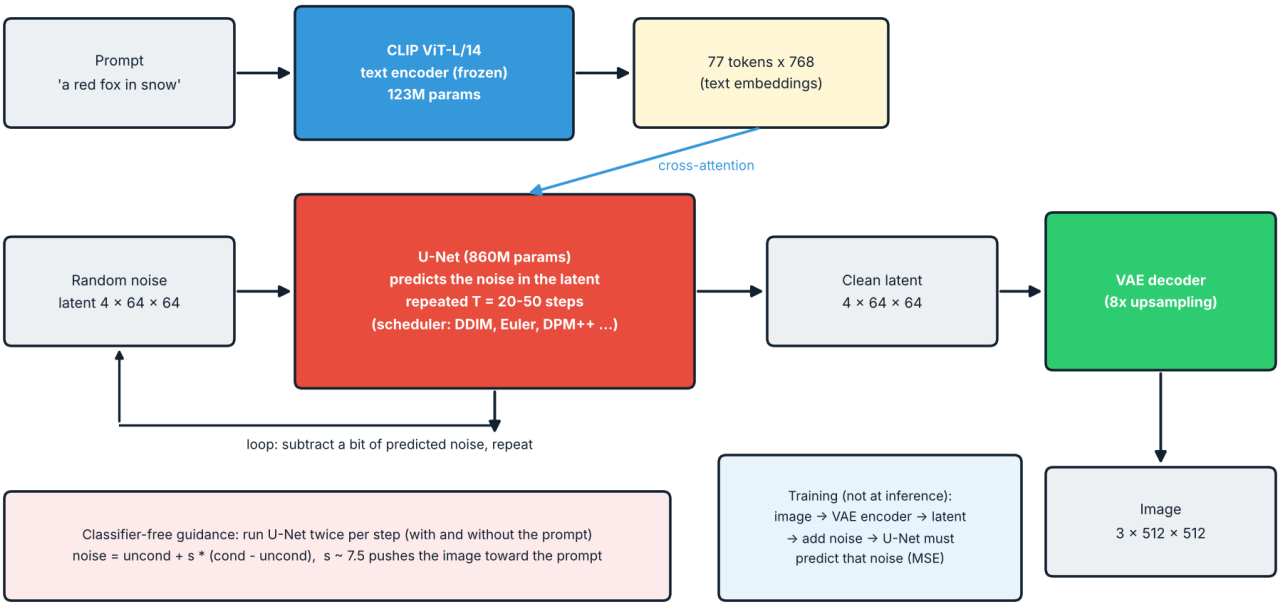
```
DDPM step: x_{t-1} = 1/sqrt(α_t) · ( x_t - (1 - α_t)/sqrt(1 - ᾱ_t) · ε_θ(x_t, t) ) + σ_t · z
DDIM step (deterministic): x0_hat = ( x_t - sqrt(1 - ᾱ_t) · ε_θ ) / sqrt(ᾱ_t); x_{t-1} = sqrt(ᾱ_{t-1}) · x0_hat + sqrt(1 - ᾱ_{t-1}) · ε_θ
```

- DDPM needs the full 1000 steps because each step is a small stochastic update. **DDIM** reinterprets the same trained model as a deterministic process, so 20 to 50 steps are enough, and the same starting noise always gives the same image.
- Modern samplers (Euler, Euler ancestral, DPM-Solver++, UniPC) are better ODE / SDE solvers that reach good quality in 15 to 30 steps. Distilled models (LCM, turbo, lightning) go down to 1 to 4 steps.

Sampler	Deterministic?	Typical steps	Note
DDPM	No	1000	Original; slow
DDIM	Yes (η = 0)	20 to 50	Fast; reproducible; enables inversion
Euler / Euler a	Euler: yes, 'a' (ancestral): no	20 to 40	Ancestral adds fresh noise each step: more variety, does not converge to one image
DPM-Solver++ / UniPC	Mostly yes	15 to 25	High-order solvers, good quality in few steps
LCM / Turbo / Lightning	Distilled	1 to 8	Student trained to skip steps; some loss in diversity / detail

4.3 Architecture of Stable Diffusion v1.x

Stable Diffusion v1.x: text-to-image at inference



The system has **three separately trained parts**: the VAE (compress / decompress), the text encoder (understand the prompt) and the U-Net (the generator). Roughly 860M (U-Net) + 123M (text encoder) + 84M (VAE) parameters *(from the model card and paper; the VAE size is approximate)*.

Part 1: the VAE (autoencoder)

- Encoder downsamples 8× in each side: 512×512×3 → 64×64×4 latent. Decoder reverses it. It is a convolutional autoencoder with residual blocks and some attention at the lowest resolution.
- Trained **once, before and separately from** the diffusion model, with: reconstruction loss (L1 / L2), **LPIPS perceptual loss** (sharp textures instead of blur), **patch-based adversarial loss** (a GAN discriminator on image patches) and a tiny KL term (weight about 1e-6) that only keeps the latent

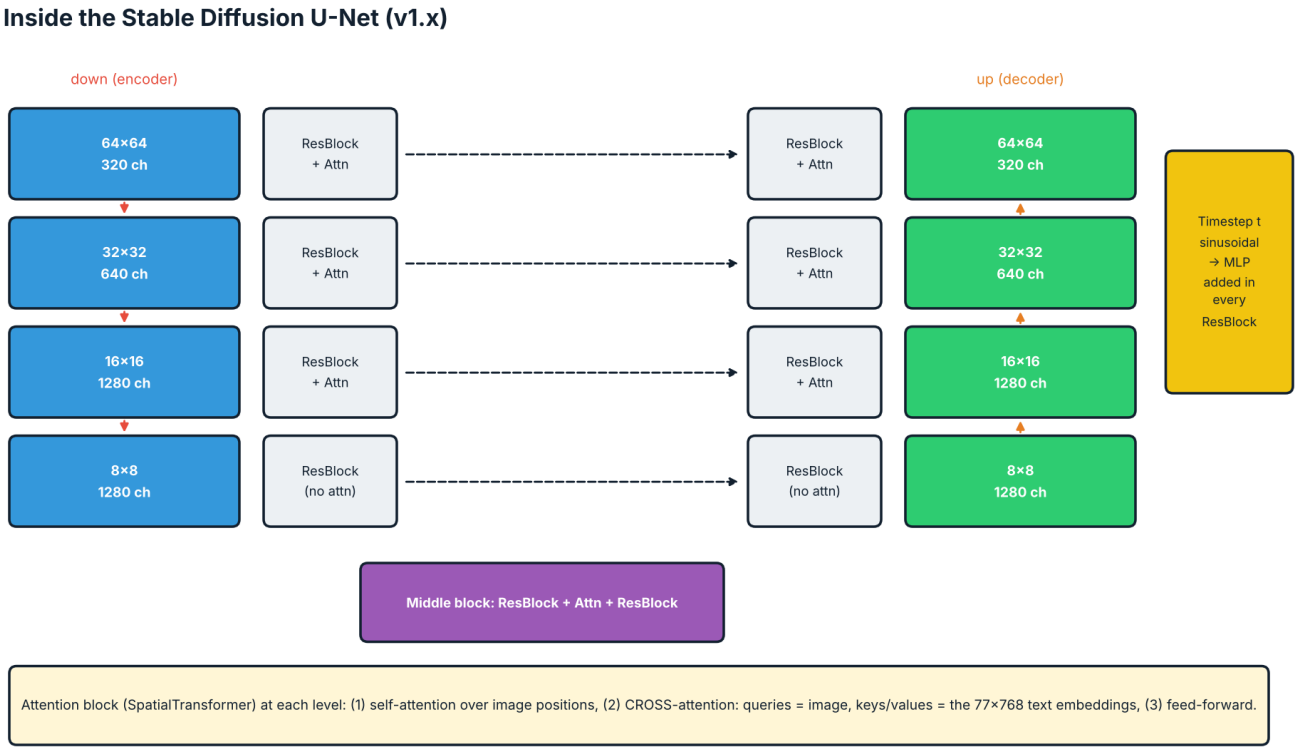
scale reasonable. Details of this are in Chapter 2.

- Latents are multiplied by a **scale factor 0.18215** before diffusion so that they have about unit variance (the diffusion noise assumes that scale).
- Limits: the VAE is a bottleneck for fine detail. Small text, faces at low resolution and tiny structures can already be damaged by 8× compression (hence the improved VAEs in SDXL and SD3).

Part 2: the text encoder

- **CLIP ViT-L/14 text Transformer, frozen** (v1.x). The prompt is tokenised and padded to 77 tokens; the output is a sequence of 77 vectors of size 768. Not a single vector: every token keeps its own embedding so the U-Net can attend to individual words.
- Frozen because it already understands language and images jointly (trained contrastively, see CLIP in the other notes). Limit: CLIP is weak at counting, spatial relations and rendering text, which shows in generated images.
- Prompts longer than 77 tokens are truncated (or chunked by tools).

Part 3: the U-Net denoiser



- Input: noisy latent z_t (4×64×64), timestep t, text embeddings. Output: predicted noise, same shape as the input (4×64×64). Same in and out shape is why a U-Net fits.
- **Levels:** resolutions 64, 32, 16, 8 with channels 320, 640, 1280, 1280 (multipliers 1, 2, 4, 4). Each level has ResNet blocks (2 on the way down, 3 on the way up); attention blocks at the first three levels and in the middle block *(from the original configuration)*.
- **Timestep embedding:** t becomes a sinusoidal vector (like Transformer positions), passes through a small MLP and is added inside every ResNet block, so every block knows how noisy the input is.
- **Self-attention** inside the attention block lets all positions in the latent exchange information (global layout). **Cross-attention** injects the text.
- Skip connections (concatenation) as in any U-Net.

```
Cross-attention: Attention(Q, K, V) = softmax( Q Kᵀ / sqrt(d) ) · V Q = W_Q · (image features), K = W_K · τ(y), V = W_V · τ(y)
```

- Queries come from the image latent (4,096 positions at the 64×64 level), keys and values come from the 77 text tokens. The attention matrix is 4,096 × 77: **each image position decides which words matter to it**. This is where 'a red fox' makes the fox region red.

Classifier-free guidance (CFG)

- Training: with probability about 10% the text is replaced by an empty prompt, so one network learns both conditional and unconditional prediction.

- Sampling: run the U-Net twice per step (with the prompt and with the empty prompt; in practice batched together) and **extrapolate** away from the unconditional prediction toward the conditional one.
- Guidance scale s (default about 7.5). s = 1: plain conditional. Higher s: follows the prompt more strongly but with less diversity and, if too high, oversaturated, artefact-prone images. Cost: doubles the U-Net work per step.

```
ε_guided = ε_uncond + s · ( ε_cond - ε_uncond )
```

Worked example: guidance

For one latent value: ε_uncond = 0.20, ε_cond = 0.50, s = 7.5. ε_guided = 0.20 + 7.5×(0.30) = **2.45**. The guided prediction is far outside the range of either prediction: the model is pushed strongly in the 'what the prompt adds' direction.

Cost of one image: 50 steps × 2 passes = 100 U-Net evaluations (batched as 50 passes with batch 2).

Training data and cost

- Trained on a subset of **LAION-5B** (hundreds of millions of image-text pairs filtered by resolution, aesthetic score and watermark probability), first at 256×256 and then at 512×512 *(from the model description)*.
- Training cost was in the order of hundreds of thousands of GPU-hours. We use pretrained weights and adapt them (LoRA, DreamBooth, textual inversion) instead of training from scratch.

4.4 What you can build on top

Technique	What it does	How
Image-to-image (SDEdit)	Edit or restyle a given image	Encode the image, add noise up to step t_start (strength), then denoise with a new prompt
Inpainting / outpainting	Fill a masked region	Extra input channels for the mask and masked image; or blend latents during sampling
ControlNet	Control layout (edges, pose, depth)	A trainable copy of the encoder takes the control image; its outputs are added to the frozen U-Net via zero-initialised convolutions
LoRA	Cheap fine-tuning of style or subject	Train small low-rank matrices added to the attention weights (a few MB instead of GB)
DreamBooth	Teach a specific subject / person	Fine-tune with a few images and a rare token; prior-preservation loss avoids forgetting
Textual inversion	Learn a new word	Optimise only a new token embedding; the model stays frozen
IP-Adapter	Image prompts	Extra cross-attention from image embeddings
Negative prompt	Steer away from things	Used in place of the empty prompt in CFG

4.5 Versions and what changed

Model	U-Net	Text encoder(s)	Other key changes
SD 1.4 / 1.5	860M parameters	CLIP ViT-L/14, context dim 768	512×512, latent 4×64×64; epsilon prediction
SD 2.0 / 2.1	865M parameters	OpenCLIP ViT-H, context dim 1024	768×768 model uses v-prediction; new training data filtering
SDXL (2023)	2.6B parameters; transformer blocks per level [0, 2, 10]; drops the lowest 8× level	OpenCLIP ViT-bigG + CLIP ViT-L, context dim 2048 (817M parameters total)	Size and crop conditioning; multi-aspect training near 1024×1024; improved VAE trained from scratch; optional **refiner** model on the first 200 noise scales
SD 3 (2024)	**MMDiT** Transformer replaces the U-Net; 450M to 8B parameters in the study	Two CLIP models + T5 (4.7B)	Separate weights for image and text tokens joined in attention; **rectified flow** training; improved autoencoder

- *Numbers for SDXL and SD3 from the SDXL paper and the Stability AI SD3 research page: 2.6B vs 860M U-Net, text encoders 817M total, 8B model, 24GB VRAM / 34 s for 1024×1024 with 50 steps (the company's early unoptimised test).*
- **Why SDXL's U-Net moves compute to lower resolutions:** it removes the 8× level and puts 10 transformer blocks at the 4× level, because attention is cheaper on small grids and the semantics matter more there.
- **Why SD3 changes the backbone:** Transformers scale predictably with size, and joint attention over text and image tokens (with separate weights per modality) improves prompt understanding and text rendering over cross-attention alone.
- **Rectified flow** connects data and noise by a straight line, $x_t = (1 - t)\cdot x_0 + t\cdot\varepsilon$, and trains the network to predict the constant velocity $(\varepsilon - x_0)$. Straighter paths allow fewer sampling steps.

```
Rectified flow: x_t = (1 - t) x_0 + t ε loss = || v_θ(x_t, t) - ( ε - x_0 ) ||² v-prediction
(SD 2.x): v = sqrt(ᾱ_t)·ε - sqrt(1 - ᾱ_t)·x_0
```

4.6 How to evaluate Stable Diffusion

- **FID** on a standard prompt set (for example COCO captions) against real images: overall realism and diversity. Same cautions as in Chapter 3.
- **CLIP score:** cosine similarity between the CLIP embedding of the image and of the prompt (a prompt-adherence proxy). Limits: CLIP is blind to counting and relations, and models can be tuned to the metric.
- **Human preference:** side-by-side comparisons, ELO scores; learned preference models (PickScore, HPS, ImageReward) approximate this.
- **Compositional benchmarks:** GenEval, T2I-CompBench, DrawBench measure objects, counts, colours, positions and attributes.
- **Diversity:** LPIPS between images from the same prompt, precision / recall in feature space.
- **Also:** latency per image and steps, memory use, safety and bias audits, memorisation checks (does it reproduce training images?).
- **Autoencoder alone:** rFID, PSNR, SSIM of reconstruction (SDXL paper reports rFID 4.4 for its VAE, 5.0 for the SD 1.x VAE *(from the SDXL paper)*).

4.7 Failure modes and fixes

Problem	Cause	Fix
Wrong object counts, bad spatial relations	CLIP text encoder and cross-attention limits	Stronger text encoders (T5, SDXL, SD3), layout control (ControlNet), prompt engineering
Garbled text in images	Weak text encoding + latent compression	SD3-style joint attention, T5 encoder
Hands and fine detail	Small structures lost in 8× latent, data variety	Bigger / better VAE, higher resolution, refiner, inpainting
Washed-out or mid-grey images	Noise schedule does not reach zero SNR	Zero-terminal-SNR rescaling, v-prediction
Over-saturated, harsh images	CFG scale too high	Lower guidance (5 to 8), dynamic thresholding
Slow generation	Many steps × 2 passes for CFG	Better samplers, distilled models, fewer steps, FP16 + efficient attention
Memorised training images, bias	Training data duplicates and web biases	Deduplication, safety filters, careful fine-tuning data
Mode collapse	Not a typical issue for diffusion	Diffusion models cover the data distribution well; low diversity usually means too high CFG or over-distilled models

5. Comparison and question bank

5.1 When to use which

Need	Choose	Reason
Pixel-accurate labels in an image	U-Net / nnU-Net	Dense prediction with precise boundaries and small data
Compact learned representation, anomaly detection	VAE	Explicit encoder, likelihood bound, smooth latent
Fast, sharp generation with a narrow domain (faces)	GAN (StyleGAN)	Single pass; high fidelity; editable latent space
Flexible, diverse, text-guided generation	Diffusion (Stable Diffusion)	Stable training, great coverage, easy conditioning
Pre-compress images for another generator	VAE / VQGAN	Makes the second model cheap (latent diffusion)

5.2 Question bank with short answers

Why does U-Net have skip connections?

The decoder loses spatial detail through pooling. Skips pass high-resolution encoder features to the decoder so it can localise boundaries while using the deep features for context.

Skip by concatenation vs addition?

Concatenation keeps both sources and lets the next layer learn how to mix them. Addition (ResNet) forces the same channel count and mixes them directly.

Why Dice loss instead of cross-entropy?

Dice measures overlap relative to object size, so it is not dominated by the large background. Often combined with cross-entropy for stable gradients.

Dice vs IoU?

Same idea, Dice = 2·IoU/(1 + IoU). Dice is always at least as large as IoU and is more forgiving; IoU punishes errors more.

Why can pixel accuracy mislead?

With 9% foreground, predicting mostly background still gives 90%+ accuracy. Use Dice, IoU, sensitivity.

What does an attention gate do?

It computes a 0 to 1 map from the skip features and a gating signal from the coarser decoder level, and multiplies the skip features by it, suppressing irrelevant regions.

U-Net++ vs U-Net?

Nested dense skip pathways reduce the semantic gap between encoder and decoder maps and add deep supervision. More parameters, better accuracy in many cases, can be pruned at inference.

What is nnU-Net?

A self-configuring pipeline that sets patch size, depth, loss and training recipe from the dataset. It shows that the recipe matters more than new blocks.

Why Transformers in U-Net?

Global context that convolutions lack. But they need more data or pretraining, so on small datasets a tuned CNN U-Net often wins.

What is the reparameterisation trick?

Write $z = \mu + \sigma \cdot \varepsilon$ with $\varepsilon \sim N(0, I)$. The sample becomes a differentiable function of μ and σ, so gradients pass through.

What is the ELBO?

A lower bound on $\log p(x)$: expected reconstruction log-likelihood minus KL between the encoder's distribution and the prior. Maximising it trains the VAE.

Why are VAE images blurry?

A Gaussian decoder with MSE predicts the average of plausible outputs, and the KL term makes posteriors overlap. Fix with perceptual / adversarial loss or by using the VAE only as a compressor.

What is posterior collapse and how do you fix it?

The encoder ignores x and q(z|x) equals the prior; KL is near 0. Fix with KL annealing, free bits, weaker decoder, lower β, or discrete latents (VQ-VAE).

β-VAE?

Multiplies the KL by $\beta > 1$ to encourage disentangled latents, at the cost of reconstruction quality.

VQ-VAE vs VAE?

Discrete codebook latent instead of Gaussian, trained with codebook and commitment losses and a straight-through gradient; no KL, so no posterior collapse, but codebook collapse is possible.

What is the GAN objective doing?

With an optimal discriminator the generator minimises the Jensen-Shannon divergence between data and model distributions.

Why the non-saturating generator loss?

The minimax loss has vanishing gradient when D rejects fakes confidently. -log D(G(z)) gives strong gradients exactly then.

Why Wasserstein?

It stays informative when real and fake distributions do not overlap, so gradients do not vanish, and the critic loss correlates with quality.

How does WGAN-GP enforce the Lipschitz constraint?

It penalises the critic's gradient norm to be 1 at random points between real and fake samples ($\lambda = 10$), replacing weight clipping.

What is mode collapse?

The generator produces only a few kinds of output. Caused by each sample being judged independently, and by G chasing whatever D currently likes.

How do you detect and fix mode collapse?

Detect: sample grids, interpolation, low LPIPS diversity, low recall, mode counting. Fix: WGAN-GP or hinge, spectral norm, minibatch-stddev or packing, unrolling, TTUR, augmentation, diversity regularisers, conditioning.

What is FID and what are its limits?

Fréchet distance between Gaussians fitted to Inception features of real and generated images. Biased for few samples, ImageNet-specific, blind to some failures; use ≥10k samples and a standard implementation.

Inception Score vs FID?

IS only looks at generated images (class confidence and class variety). FID compares against real data, so it detects missing diversity and shifts better.

Why diffusion instead of GAN?

Stable training, excellent coverage of modes, flexible conditioning. Price: many sampling steps (mitigated by better samplers and distillation).

What does the network predict in diffusion?

The noise ε added to the clean sample (sometimes v or x_0). The loss is MSE between true and predicted noise.

Why diffuse in latent space?

A 48× smaller representation makes both training and sampling far cheaper while keeping perceptual quality, using an autoencoder trained once.

How does text enter Stable Diffusion?

A frozen CLIP text encoder gives 77 token embeddings; cross-attention layers in the U-Net use them as keys and values with image features as queries.

What is classifier-free guidance?

Train with random prompt dropout, then at sampling extrapolate: ε_uncond + s(ε_cond - ε_uncond). Larger s follows the prompt more but reduces diversity.

DDPM vs DDIM?

Same trained model. DDPM samples stochastically with ~1000 steps, DDIM uses a deterministic update with 20 to 50 steps.

What changed in SDXL and SD3?

SDXL: 3× larger U-Net, two text encoders, size/crop conditioning, refiner, new VAE. SD3: Transformer (MMDiT) with joint text-image attention, rectified flow, three text encoders.

What is LoRA?

Low-rank updates added to attention weights, trained instead of the full model: small files, fast fine-tuning.

6. Sources and original figures

Open these to see the **original figures** from the papers (U-Net Figure 1, Attention U-Net Figure 2, U-Net++ Figure 1, the VAE and GAN diagrams, the Latent Diffusion architecture figure, the SDXL and SD3 diagrams). Links point to arXiv abstract pages; check licences before reusing figures in your own material.

Topic	Authors / year	Link
U-Net	Ronneberger, Fischer, Brox 2015	https://arxiv.org/abs/1505.04597
3D U-Net	Çiçek et al. 2016	https://arxiv.org/abs/1606.06650
V-Net	Milletari et al. 2016	https://arxiv.org/abs/1606.04797
Attention U-Net	Oktay et al. 2018	https://arxiv.org/abs/1804.03999
U-Net++	Zhou et al. 2018	https://arxiv.org/abs/1807.10165
UNet 3+	Huang et al. 2020	https://arxiv.org/abs/2004.08790
nnU-Net	Isensee et al. 2018	https://arxiv.org/abs/1809.10486
TransUNet	Chen et al. 2021	https://arxiv.org/abs/2102.04306
Swin-UNet	Cao et al. 2021	https://arxiv.org/abs/2105.05537
UNETR	Hatamizadeh et al. 2021	https://arxiv.org/abs/2103.10504
U-Net review (variants, theory)	Siddique et al. 2020	https://arxiv.org/abs/2011.01118
U-Net review (2025)	Comprehensive review	https://arxiv.org/abs/2502.06895
VAE	Kingma and Welling 2013	https://arxiv.org/abs/1312.6114
VQ-VAE	van den Oord et al. 2017	https://arxiv.org/abs/1711.00937
VQGAN	Esser et al. 2021	https://arxiv.org/abs/2012.09841
Lagging inference and posterior collapse	He et al. 2019	https://arxiv.org/abs/1901.05534
GAN	Goodfellow et al. 2014	https://arxiv.org/abs/1406.2661
DCGAN	Radford et al. 2015	https://arxiv.org/abs/1511.06434
Wasserstein GAN	Arjovsky et al. 2017	https://arxiv.org/abs/1701.07875
WGAN-GP	Gulrajani et al. 2017	https://arxiv.org/abs/1704.00028
LSGAN	Mao et al. 2016	https://arxiv.org/abs/1611.04076
Spectral normalisation	Miyato et al. 2018	https://arxiv.org/abs/1802.05957
Unrolled GAN	Metz et al. 2016	https://arxiv.org/abs/1611.02163
Pix2Pix	Isola et al. 2016	https://arxiv.org/abs/1611.07004
CycleGAN	Zhu et al. 2017	https://arxiv.org/abs/1703.10593
Progressive GAN	Karras et al. 2017	https://arxiv.org/abs/1710.10196
StyleGAN	Karras et al. 2018	https://arxiv.org/abs/1812.04948
BigGAN	Brock et al. 2018	https://arxiv.org/abs/1809.11096
FID and TTUR	Heusel et al. 2017	https://arxiv.org/abs/1706.08500
Inception Score (Improved techniques)	Salimans et al. 2016	https://arxiv.org/abs/1606.03498
KID	Binkowski et al. 2018	https://arxiv.org/abs/1801.01401
Improved precision and recall	Kynkäänniemi et al. 2019	https://arxiv.org/abs/1904.06991
Pros and cons of GAN evaluation measures	Borji 2018	https://arxiv.org/abs/1802.03446
DDPM	Ho et al. 2020	https://arxiv.org/abs/2006.11239
DDIM	Song et al. 2020	https://arxiv.org/abs/2010.02502
Latent Diffusion (Stable Diffusion)	Rombach et al. 2021	https://arxiv.org/abs/2112.10752

Topic	Authors / year	Link
Classifier-free guidance	Ho and Salimans 2022	https://arxiv.org/abs/2207.12598
ControlNet	Zhang et al. 2023	https://arxiv.org/abs/2302.05543
LoRA	Hu et al. 2021	https://arxiv.org/abs/2106.09685
SDXL	Podell et al. 2023	https://arxiv.org/abs/2307.01952
Stable Diffusion 3 (rectified flow transformers)	Esser et al. 2024	https://arxiv.org/abs/2403.03206
DiT	Peebles and Xie 2022	https://arxiv.org/abs/2212.09748
Stable Diffusion 3 research page	Stability AI	https://stability.ai/news/stable-diffusion-3-research-paper

Facts confirmed during this session by reading the SDXL paper, the Stability AI SD3 research page and a review of U-Net variants. The arXiv identifiers above are from memory of the standard references; if a link does not open, search the title.